

项目八股复习1

1. 实时的播放性能如何优化

1.前端优化：

1.缓冲区管理： 合理设置播放器缓冲区大小，避免频繁的重新缓冲或卡顿。预加载一小部分数据，确保播放流畅。

2.渲染优化： 使用硬件加速，减少CPU负担。优化视频渲染路径，例如使用WebGL或DirectX等技术。

3.低延迟传输协议： 优先选择低延迟的传输协议，如WebRTC、SRT（Secure Reliable Transport）等，而不是传统的HTTP Live Streaming (HLS) 或 MPEG-DASH，虽然这些也可以通过参数优化实现较低延迟。

4.播放器选择与配置： 选择高性能的播放器，并对其进行精细化配置，例如调整解码器、渲染器、缓存大小等。

5.流媒体协议优化： 根据网络环境和内容特点选择合适的流媒体协议，并优化其参数。例如，对于HLS或DASH，可以缩短切片时长，减少GOP间隔，使用CMAF等。

2.编码与压缩：

1.选择高效的编码器： 使用H.264、H.265 (HEVC) 等高效视频编码标准，甚至未来的AV1、VVC等，它们能在相同画质下获得更小的码流。

2.码率控制： 根据网络带宽和设备性能，采用VBR（可变码率）或CBR（恒定码率）进行编码，并在保证画质的前提下，尽可能降低码率。

3.GOP（Group of Pictures）设置： 减小GOP长度可以降低延时，但会略微增加码率。在低延迟场景下，GOP长度通常设置在1-2秒。

4.B帧优化： 减少B帧或禁用B帧（如果对延时要求极高），因为B帧依赖于前后帧，会增加编解码延迟。

5.Profile和Level： 根据目标设备和平台，选择合适的编码Profile和Level，确保兼容性和性能。

6.网络传输优化：

1.**CDN加速**： 使用内容分发网络（CDN）将内容分发到离用户最近的节点，减少传输距离，降低延迟。

2.**QoS（Quality of Service）**： 在网络设备上配置QoS策略，优先保障媒体流的传输带宽和优先级。

3.**FEC（Forward Error Correction）**： 在高丢包率网络环境下使用FEC，减少重传，提高传输效率。

4.**拥塞控制算法**： 使用更适应实时流媒体的拥塞控制算法（例如Google的BBR），而不是传统的TCP Reno/Cubic。

5.**UDP传输**： 对于对延迟要求极高的场景，可以考虑基于UDP的传输协议，如RTP/RTCP，并在此基础上构建可靠传输机制。

3.后端服务优化：

1.转码服务优化： 如果需要实时转码，确保转码服务器性能强大，并使用硬件加速。

2.存储优化： 采用SSD存储媒体文件，提高I/O性能。

3.缓存策略： 对热门内容进行缓存，减少源站压力。

2. FFmpeg的解码流程

1. 初始化AVFormatContext:

通过 `avformat_open_input()` 打开输入文件或流。

通过 `avformat_find_stream_info()` 读取文件头部信息，解析出媒体流的数量、类型、时长等元数据。

2. 查找并打开解码器:

遍历 `AVFormatContext` 中的所有 `AVStream`（例如视频流、音频流）。

对于每个流，根据其 `codecpar`（编码器参数）中的 `codec_id`，使用 `avcodec_find_decoder()` 查找对应的解码器。

使用 `avcodec_alloc_context3()` 分配一个 `AVCodecContext`，并用 `avcodec_parameters_to_context()` 将 `codecpar` 拷贝到 `AVCodecContext` 中。

使用 `avcodec_open2()` 打开解码器。

3. 循环读取数据包并解码:

读取数据包： 使用 `av_read_frame()` 从输入文件中读取一个压缩数据包（`AVPacket`）。`AVPacket` 包含了一帧或多帧编码数据，通常是压缩过的。

发送数据包到解码器： 将读取到的 `AVPacket` 发送给解码器，使用 `avcodec_send_packet()`。

接收解码帧： 循环调用 `avcodec_receive_frame()` 从解码器中接收解码后的原始帧（`AVFrame`）。

对于视频流，`AVFrame` 包含未经压缩的像素数据，通常是YUV格式。

对于音频流，`AVFrame` 包含未经压缩的PCM音频数据。

处理解码帧： 对 `AVFrame` 进行后续处理，例如：

视频帧： 进行缩放、色彩空间转换、渲染显示等。

音频帧： 进行重采样、音量调整、播放等。

释放数据包： 使用 `av_packet_unref()` 释放 `AVPacket`。

4. 循环结束与资源释放：

当 `av_read_frame()` 返回 `AVERROR_EOF` 或其他错误时，表示文件读取结束。

继续调用 `avcodec_receive_frame()`，直到解码器清空所有缓存的帧（这对于有B帧的视频尤为重要）。

使用 `avcodec_close()` 关闭解码器。

使用 `avformat_close_input()` 关闭输入文件。

使用 `avcodec_free_context()` 释放 `AVCodecContext`。

使用 `av_frame_free()` 释放 `AVFrame`。

使用 `av_packet_free()` 释放 `AVPacket`。

FFmpeg解码的核心是： `AVFormatContext` (文件/流封装) -> `AVPacket` (压缩数据包) -> `AVCodecContext` (解码器) -> `AVFrame` (原始数据帧)。

3. 数据库和Redis分别实现了什么

数据库（以关系型数据库为例）主要实现：

持久化存储： 数据存储在磁盘上，即使系统重启也不会丢失。这是数据库最核心的功能。

数据结构化与管理： 通过表、行、列、索引等结构化方式存储数据，并提供SQL（或其他查询语言）进行数据查询、插入、更新、删除等操作。

事务支持（ACID）： 保证数据的原子性、一致性、隔离性、持久性。这意味着数据库能够确保并发操作的正确性，并在系统崩溃时保持数据完整性。

数据完整性约束： 通过主键、外键、唯一约束、非空约束等机制，强制保证数据的合法性和一致性。

数据安全与权限控制： 提供用户认证、授权机制，控制不同用户对数据的访问权限。

复杂查询与分析： 支持复杂的SQL查询，包括连接、分组、聚合、子查询等，能够进行深入的数据分析和报表生成。

备份与恢复： 提供完善的备份和恢复机制，保障数据安全。

高可用与可扩展性： 通过主从复制、分片等技术实现高可用和水平扩展。

Redis主要实现：

高速缓存： 作为内存数据库，Redis将数据存储于RAM中，读写速度极快，远超磁盘存储的传统数据库。这是Redis最主要的应用场景，用于缓解数据库压力，加速数据访问。

丰富的数据结构： 除了简单的键值对外，Redis还支持多种复杂的数据结构，如字符串（String）、哈希（Hash）、列表（List）、集合（Set）、有序集合（Sorted Set），以及发布/订阅（Pub/Sub）、地理空间索引（Geo）等。这些数据结构使得Redis能够处理各种复杂的业务场景。

持久化支持： Redis提供了RDB（快照）和AOF（只追加文件）两种持久化机制，可以将内存中的数据定期或实时写入磁盘，以防止数据丢失。但其主要优势仍是内存读写速度。

分布式锁： Redis的原子性操作可以用于实现分布式锁，解决分布式系统中的并发问题。

消息队列： 通过列表（List）或发布/订阅（Pub/Sub）功能，Redis可以实现简单的消息队列，用于异步处理任务或组件间通信。

排行榜与计数器： 有序集合（Sorted Set）非常适合实现排行榜功能，而原子递增/递减操作则非常适合计数器。

会话管理： 存储用户会话信息，实现快速的用户登录和状态管理。

实时数据处理： 由于其极高的读写性能，Redis可以用于实时分析、实时统计等场景。

4. 自适应码率怎么实现的

1. 视频编码与多版本生成：

转码： 原始视频文件被编码成多个不同码率和分辨率的版本。例如，一个视频可能被编码成 360p (低码率)、720p (中码率)、1080p (高码率) 等多个版本。

切片/分段： 每个版本的视频流都会被分割成若干个小的、独立的媒体片段（Segment 或 Chunk）。这些片段通常是几秒钟的时长。

封装： 这些片段会以特定的流媒体协议进行封装，例如：

HLS (HTTP Live Streaming)： 由Apple推广，使用MPEG-TS (Transport Stream) 作为媒体片段格式，通过M3U8播放列表文件来描述所有可用版本和片段信息。

MPEG-DASH (Dynamic Adaptive Streaming over HTTP)： 国际标准，使用MP4作为媒体片段格式，通过MPD (Media Presentation Description) 文件来描述。

Microsoft Smooth Streaming、Adobe HDS (HTTP Dynamic Streaming) 等也有类似机制。

2. 播放器端逻辑：

下载清单文件： 播放器首先下载自适应码率流的清单文件（例如HLS的M3U8文件或DASH的MPD文件）。这个文件包含了所有可用版本（码率、分辨率）的元数据，以及每个版本的媒体片段URL。

初始码率选择： 播放器根据初始的网络带宽检测、设备能力或用户设置，选择一个合适的初始码率版本开始播放。

实时带宽检测： 这是自适应码率的核心。播放器在播放过程中持续监测用户的网络带宽。这通常通过测量已下载媒体片段的下载时间、下载速度等指标来实现。

缓冲区管理： 播放器会维护一个媒体缓冲区。当缓冲区数据充足时，意味着网络状况良好，可以尝试提升码率；当缓冲区数据不足（甚至有卡顿风险）时，意味着网络状况不佳，需要降低码率。

码率切换算法： 播放器根据实时带宽检测结果和缓冲区状态，运行一个码率切换算法。这个算法会决定何时以及切换到哪个码率版本。

向上切换： 如果网络带宽持续良好，并且缓冲区保持在健康水平之上，播放器会尝试切换到更高码率的版本，以提供更好的画质。

向下切换： 如果网络带宽下降，或者缓冲区开始耗尽，播放器会立即切换到更低码率的版本，以避免卡顿。

无缝切换： 为了确保切换过程对用户是无感知的，自适应码率系统会确保不同码率版本的媒体片段在时间轴上是对齐的。当切换发生时，播放器会从当前正在播放的片段结束处开始下载新码率版本的对应片段。

反馈与调整： 播放器会持续迭代这个过程，根据实时的网络和播放情况进行调整。

5. 数据一致性怎么保证

数据一致性在分布式系统和高并发场景中是一个非常重要且复杂的问题。它指的是在多个副本或多个节点之间，或者在多个操作过程中，数据保持逻辑上的一致性。保证数据一致性的方法有很多，具体取决于所需的一致性级别和系统特点。

常见的几种数据一致性级别：

强一致性： 最严格的一致性。当一个写操作完成后，任何后续的读操作都能立即看到这个写操作的最新结果。这通常通过锁、事务或共识算法实现，但代价是高延迟和低可用性。

弱一致性： 允许数据在更新后，不立即对所有读操作可见。最终会达到一致，但需要一定时间。

最终一致性： 弱一致性的一种特殊形式。系统不保证立即看到最新数据，但在经过一段不确定的时间后，所有副本最终会达到一致状态。这是许多分布式系统（如NoSQL数据库）常用的模型，以提高可用性和性能。

读写一致性： 保证用户自己写入的数据，在后续的读操作中一定能读到。

会话一致性： 在一个会话内，用户能读到自己写入的数据，并且读到的数据是单调递增的（不会看到旧数据）。

保证数据一致性的主要方法：

1. 事务（**Transactions**）：

ACID特性： 在关系型数据库中，通过ACID（原子性、一致性、隔离性、持久性）事务来保证数据一致性。例如，当多个操作组成一个逻辑单元时，要么全部成功，要么全部失败回滚，从而避免数据处于中间不一致状态。

分布式事务： 在分布式系统中，实现分布式事务（如两阶段提交2PC、三阶段提交3PC、TCC等）来保证跨多个服务的原子性操作，但实现复杂，性能开销大。

2. 锁机制：

悲观锁： 在数据被访问时就加锁，阻止其他并发操作修改数据。适用于写操作较多、冲突概率高的场景，但并发性能较低。

乐观锁： 不加锁，而是在更新时检查数据是否被其他操作修改过。通常通过版本号（**version**）或时间戳（**timestamp**）实现。适用于读操作较多、冲突概率低的场景，并发性能较高。

分布式锁： 在分布式系统中，利用Redis、ZooKeeper等工具实现分布式锁，确保同一时刻只有一个进程能访问共享资源。

3. 消息队列（**Message Queue**）：

最终一致性：结合消息队列实现最终一致性。当一个服务完成一个操作后，发送消息到消息队列，其他服务订阅并消费消息，异步地更新自己的数据。这能解耦服务，提高系统吞吐量和可用性，但会引入延迟。

事务消息：例如RocketMQ的事务消息，可以保证消息发送和本地事务的原子性，进一步提高最终一致性的可靠性。

4. 共识算法：

Paxos / Raft：这些算法用于在分布式系统中达成一致，例如选举Leader、复制状态机等。它们能保证在部分节点故障的情况下，系统仍然能够对外提供一致的服务，但实现复杂。

5. 版本控制：

通过为数据记录版本号或时间戳，在更新时检查版本号，可以实现乐观锁，并解决冲突。

6. 读写分离与主从复制：

主从复制：主数据库负责写操作，从数据库负责读操作，主库的数据会异步或同步复制到从库。

一致性问题：异步复制会导致主从延迟，可能出现读到旧数据的问题（弱一致性）。可以通过强制读主库、延迟读、读写分离中间件等方式来缓解。

7. 数据校验与修复：

定期对数据进行校验，发现不一致的地方进行人工或自动修复。

6. 延迟双删的时候会有脏数据吗

延迟双删 是一种用于保证缓存和数据库之间数据最终一致性的策略，特别是在读多写少的场景下，用于解决缓存更新和数据库更新的并发问题。

其基本思路是：

1. 先删除缓存。
2. 再更新数据库。
3. 延迟一段时间后，再次删除缓存。

延迟双删的目的：

主要是为了应对以下并发场景：

线程A删除缓存，更新数据库；线程B读取缓存（未命中），读取旧数据库数据，写入旧缓存。如果此时线程A没有延迟双删，那么缓存中就会是旧数据。

线程**A**删除缓存，更新数据库；线程**B**读取缓存（未命中），读取新数据库数据，写入新缓存。这时候线程**A**的延迟双删是多余的，但不会有问题。

延迟双删解决的问题是：

在“删除缓存 -> 更新数据库”期间，如果有其他线程读取，读到的是数据库的旧数据，然后把旧数据写入缓存，导致缓存和数据库不一致。延迟双删就是为了再次删除这个可能被写入的旧缓存。

延迟双删会有脏数据吗？

理论上，延迟双删本身并不能完全杜绝脏数据，在某些极端并发场景下，仍然可能产生脏数据，但它极大地降低了脏数据产生的概率。

可能产生脏数据的极端场景：

假设有以下操作序列：

1. 线程**A**：删除缓存（成功）。
2. 线程**A**：更新数据库（成功，值为新值 **D_new**）。
3. 线程**B**：查询缓存（未命中）。
4. 线程**B**：查询数据库（此时线程**A**已经更新完成，查询到数据库新值 **D_new**）。
5. 线程**B**：将 **D_new** 写入缓存。
6. 线程**C**：查询缓存（命中），得到 **D_new**。
7. 线程**A**：延迟删除缓存（成功）。
8. 线程**C**：查询缓存（未命中）。
9. 线程**C**：查询数据库（此时数据库值仍为 **D_new**）。
10. 线程**C**：将 **D_new** 写入缓存。

在这个例子中，虽然线程**A**进行了延迟双删，但在线程**A**的第二次删除缓存之前，线程**B**已经将最新的数据库数据写入了缓存。当线程**A**进行第二次删除时，它删除的是线程**B**写入的最新数据。

问题出在哪里？

延迟双删主要应对的是“读旧数据写旧缓存”的情况。它无法解决“写新数据再删除新缓存”的情况。

更关键的问题在于：

数据库和缓存的更新顺序问题：无论你先删缓存后更新数据库，还是先更新数据库后删缓存，都存在并发问题。

延迟时间的不确定性：延迟多长时间是合适的？如果延迟时间太短，可能还未覆盖到写入旧缓存的请求；如果延迟时间太长，则会增加缓存和数据库数据不一致的窗口。

如何更有效地保证数据一致性？

延迟双删虽然能降低概率，但并不是银弹。更严格的一致性保证通常需要：

强一致性读写：牺牲性能，确保读写都在同一个数据源上完成。

写入时更新缓存：当数据更新时，直接更新缓存而不是删除缓存。这需要保证原子性，例如通过分布式事务或消息队列来协调数据库和缓存的更新。

结合消息队列：在更新数据库成功后，发送一条消息到消息队列。消费者异步地去删除或更新缓存。这可以实现最终一致性，并且解耦了服务。

使用缓存淘汰策略（**TTL**）：为缓存设置合理的过期时间，即使偶尔出现不一致，也能在一段时间后自动修复。

7. 如果线程池的线程达到了上限怎么办

如果线程池的线程达到了 `maximumPoolSize`，并且 `workQueue` 也已满，这意味着线程池已经饱和，无法再接受新的任务。这时，线程池会根据其配置的拒绝策略来处理新的任务。

`ThreadPoolExecutor` 提供了四种内置的拒绝策略：

1. `AbortPolicy` (默认策略)：

行为：直接抛出 `RejectedExecutionException` 运行时异常。

影响：调用者会捕获到这个异常，需要自行处理，例如记录日志、返回错误信息给用户。

使用场景：适合对任务处理失败比较敏感，需要明确通知调用者的场景。

2. `CallerRunsPolicy`：

行为：不抛弃任务，也不抛出异常，而是将这个任务回退到提交任务的线程（即调用 `execute()` 方法的线程）去执行。

影响：提交任务的线程会阻塞，直到这个任务执行完成。这会减慢提交任务的速率，从而给线程池和队列“喘息”的机会，避免任务提交过快导致系统崩溃。

使用场景：适合希望系统缓慢降级，而不是直接拒绝任务的场景。它可以在一定程度上防止系统过载。

3. `DiscardPolicy`：

行为：直接丢弃当前提交的任务，不抛出任何异常。

影响：调用者不会收到任何反馈，任务就悄无声息地丢失了。

使用场景：适合那些可以接受少量任务丢失，对任务结果不那么敏感，或者任务量非常大以至于丢弃部分任务可以保证系统整体稳定性的场景（例如日志、监控数据等）。

4. `DiscardOldestPolicy`:

行为：丢弃工作队列中最老（等待时间最长）的任务，然后尝试重新提交当前任务。

影响：新任务有机会被执行，但最老的任务会被丢弃。这适用于那些“新”的任务比“旧”的任务更重要，或者旧任务已经失去时效性的场景。

使用场景：实时性要求较高的场景，例如实时数据处理，宁愿处理新数据而丢弃旧数据。

除了内置的拒绝策略，我们还可以自定义拒绝策略：

实现 `RejectedExecutionHandler` 接口，并重写其 `rejectedExecution()` 方法。在自定义策略中，可以实现更复杂的逻辑，例如：

记录日志并报警：将被拒绝的任务信息记录下来，并通过邮件、短信等方式通知管理员。

持久化任务：将任务保存到数据库或消息队列中，等待系统恢复后再处理。

优雅降级：返回一个默认值或一个失败结果给调用者。

重试机制：在一段时间后再次尝试提交任务。

8. 设计模式在项目当中的使用

1. 创建型模式

用于对象的创建，将对象的创建与使用分离。

单例模式：

使用场景：确保一个类只有一个实例，并提供一个全局访问点。例如，日志管理器、数据库连接池、配置管理器、线程池、缓存管理等。

项目示例：全局唯一的配置对象 `ConfigManager`，确保所有模块都使用同一份配置。

工厂方法模式：

使用场景： 定义一个用于创建对象的接口，让子类决定实例化哪一个类。延迟了实例化到子类。

项目示例： 根据不同的订单类型（普通订单、VIP订单、团购订单）创建不同的订单处理器 `OrderProcessor`。

抽象工厂模式：

使用场景： 提供一个接口，用于创建相关或依赖对象的家族，而无需指定具体类。

项目示例： 创建不同操作系统的UI组件（按钮、文本框），例如 `WindowsFactory` 创建 `WinButton` 和 `WinTextBox`，`MacFactory` 创建 `MacButton` 和 `MacTextBox`。

建造者模式：

使用场景： 将一个复杂对象的构建与其表示分离，使得同样的构建过程可以创建不同的表示。

项目示例： 构建复杂的用户查询条件 `UserQueryBuilder`，包含姓名、年龄、性别、地址等多个可选条件。

原型模式：

使用场景： 通过复制现有对象来创建新对象，而不是从头创建，尤其当对象的创建成本较高时。

项目示例： 复制复杂的报表对象或配置对象，避免重复创建。

2. 结构型模式

用于将类或对象组合成更大的结构，以实现新的功能。

适配器模式：

使用场景： 允许接口不兼容的类协同工作。将一个类的接口转换成客户希望的另一个接口。

项目示例： 集成第三方API，当第三方API的接口与我们系统不兼容时，通过适配器将其转换。例如，将旧的支付接口适配到新的支付网关。

装饰者模式：

使用场景： 动态地给一个对象添加一些额外的职责。相比较继承，装饰者更加灵活。

项目示例： 为一个核心功能（如数据加密、日志记录、权限校验）动态添加或移除其他功能，例如对 `IUserService` 接口进行 `LogDecorator` 或 `AuthDecorator` 包装。

代理模式：

使用场景： 为另一个对象提供一个替身或占位符，以控制对这个对象的访问。

组合模式：

使用场景： 将对象组合成树形结构以表示“部分-整体”的层次结构。允许用户对单个对象和组合对象使用相同的操作。

项目示例： 文件系统（文件夹包含文件和子文件夹）、公司组织结构（部门包含员工和子部门）、菜单系统。

外观模式：

使用场景： 为子系统中的一组接口提供一个统一的入口。定义一个高层接口，使子系统更容易使用。

项目示例： 封装复杂的订单处理流程（库存检查、支付、物流、积分），对外只暴露一个简单的 `OrderService.placeOrder()` 方法。

桥接模式：

使用场景： 将抽象与实现分离，使它们可以独立变化。

项目示例： 允许UI元素（如按钮、下拉菜单）与其底层图形库（如AWT、Swing、Qt）独立变化。

享元模式：

使用场景： 尽量减少内存使用，通过共享相同对象而不是每次都创建新对象。

项目示例： 文本编辑器中的字符对象，池化对象（如数据库连接池）。

3. 行为型模式

用于描述对象之间的交互以及它们如何协作完成任务。

策略模式：

使用场景： 定义一系列的算法，并将每一个算法封装起来，使它们可以相互替换。算法的变化独立于使用算法的客户。

项目示例： 不同的支付方式（支付宝支付、微信支付、银行卡支付）、不同的排序算法、不同的优惠计算策略。

观察者模式：

使用场景： 定义对象间的一种一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并被自动更新。

项目示例： 事件通知系统、MVC框架中的模型-视图更新、消息队列。

模板方法模式：

使用场景： 定义一个操作中的算法的骨架，而将一些步骤延迟到子类中。模板方法使得子类可以在不改变算法结构的情况下，重新定义该算法的某些特定步骤。

项目示例： 订单处理流程（创建订单、支付、发货、通知），其中支付方式或发货方式可能在子类中实现。

迭代器模式：

使用场景： 提供一种方法顺序访问一个聚合对象中各个元素，而又不暴露该对象的内部表示。

项目示例： 遍历各种集合（List、Set、Map）。

责任链模式：

使用场景： 避免请求发送者与接收者耦合在一起，让多个对象都有可能处理请求，将这些对象连成一条链，并且沿着这条链传递请求，直到有一个对象处理它为止。

项目示例： 请求过滤、审批流程（请假审批、报销审批）、日志级别处理。

命令模式：

使用场景： 将一个请求封装成一个对象，从而使你可用不同的请求对客户进行参数化；对请求排队或记录请求日志，以及支持可撤销的操作。

项目示例： 菜单操作（撤销、重做）、宏命令、事务处理。

状态模式：

使用场景： 允许一个对象在其内部状态改变时改变它的行为。对象看起来好像修改了它的类。

项目示例： 订单状态流转（待支付、已支付、待发货、已发货、已完成）、用户状态管理、电梯运行状态。

中介者模式：

使用场景： 用一个中介对象来封装一系列的对象交互。中介者使各对象不需要显式地相互引用，从而使其耦合松散，而且可以独立地改变它们之间的交互。

项目示例： 聊天室、GUI组件之间的交互。

备忘录模式：

使用场景： 在不破坏封装性的前提下，捕获一个对象的内部状态，并在该对象之外保存这个状态。这样以后就可以将该对象恢复到保存过的状态。

项目示例： 文本编辑器的撤销操作、游戏存档。

访问者模式：

使用场景： 表示一个作用于某对象结构中的各元素的操作。它使你可以在不改变各元素的类的前提下定义作用于这些元素的新操作。

项目示例： 对文档结构（段落、图片、表格）进行不同的操作（如导出为HTML、PDF）。

9. 单例怎么保证线程安全

1. 饿汉式

在程序启动时就创建实例。这种方式天生线程安全，因为实例在主程序启动前就已经创建完成。

```
#include <iostream>
class SingletonHungry {
private:
    // 私有构造函数，防止外部直接实例化
    SingletonHungry() {
        std::cout << "SingletonHungry instance created." <<
std::endl;
    }
    // 禁用拷贝构造函数和赋值运算符
    SingletonHungry(const SingletonHungry&) = delete;
    SingletonHungry& operator=(const SingletonHungry&) = delete;

public:
    // 在程序启动时就创建实例
    static SingletonHungry instance;

    // 提供全局访问点
    static SingletonHungry& getInstance() {
        return instance;
    }

    void doSomething() {
        std::cout << "SingletonHungry doing something." <<
std::endl;
    }
};

// 静态成员变量的定义和初始化
SingletonHungry SingletonHungry::instance; // 在这里创建实例

// 使用示例
// int main() {
//     SingletonHungry& s1 = SingletonHungry::getInstance();
//     SingletonHungry& s2 = SingletonHungry::getInstance();
//     s1.doSomething();
//     // s1 和 s2 是同一个实例
// }
```

优点： 线程安全，实现简单。

缺点： 非懒加载，即使不使用也会创建实例，可能造成资源浪费。

2. 懒汉式 - 静态局部变量

这是C++11及更高版本中最推荐和最简洁的线程安全懒汉式单例实现方式。它利用了C++标准（C++11起）保证静态局部变量只会被初始化一次的特性。

```
#include <iostream>
#include <thread>
#include <vector>

class SingletonMeyers {
private:
    SingletonMeyers() {
        std::cout << "SingletonMeyers instance created." <<
std::endl;
    }
    SingletonMeyers(const SingletonMeyers&) = delete;
    SingletonMeyers& operator=(const SingletonMeyers&) = delete;

public:
    static SingletonMeyers& getInstance() {
        // C++11标准保证静态局部变量的初始化是线程安全的（仅初始化一次）
        static SingletonMeyers instance;
        return instance;
    }

    void doSomething() {
        std::cout << "SingletonMeyers doing something." <<
std::endl;
    }
};

// 使用示例
// void thread_func() {
//     SingletonMeyers& s = SingletonMeyers::getInstance();
//     s.doSomething();
// }
```

```
// int main() {  
//     std::vector<std::thread> threads;  
//     for (int i = 0; i < 5; ++i) {  
//         threads.emplace_back(thread_func);  
//     }  
//     for (auto& t : threads) {  
//         t.join();  
//     }  
// }
```

优点：

线程安全： C++11标准保证静态局部变量的初始化是线程安全的。

懒加载： 只有在第一次调用 `getInstance()` 时才创建实例。

实现简洁： 代码量少，易于理解。

无额外性能开销： 初始化完成后，后续访问没有同步开销。

缺点： 无明显缺点，是现代C++中最推荐的单例实现方式。

3. 懒汉式 - `std::call_once`

`std::call_once` 是C++11提供的一个机制，它确保某个函数（这里是实例的创建）只会被调用一次，即使在多线程环境下。

```
#include <iostream>  
#include <mutex>  
#include <thread>  
#include <vector>  
  
class SingletonCallOnce {  
private:  
    SingletonCallOnce() {  
        std::cout << "SingletonCallOnce instance created." <<  
std::endl;  
    }  
    SingletonCallOnce(const SingletonCallOnce&) = delete;
```

```

SingletonCallOnce& operator=(const SingletonCallOnce&) =
delete;

static SingletonCallOnce* instance;
static std::once_flag initFlag;

// 辅助函数，用于创建实例
static void initSingleton() {
    instance = new SingletonCallOnce();
}

public:
    static SingletonCallOnce& getInstance() {
        // 确保 initSingleton() 只被调用一次
        std::call_once(initFlag, initSingleton);
        return *instance;
    }

    void doSomething() {
        std::cout << "SingletonCallOnce doing something." <<
std::endl;
    }
};

// 静态成员变量的定义和初始化
SingletonCallOnce* SingletonCallOnce::instance = nullptr;
std::once_flag SingletonCallOnce::initFlag;

// 使用示例
// void thread_func_call_once() {
//     SingletonCallOnce& s = SingletonCallOnce::getInstance();
//     s.doSomething();
// }

// int main() {
//     std::vector<std::thread> threads;
//     for (int i = 0; i < 5; ++i) {
//         threads.emplace_back(thread_func_call_once);
//     }
//     for (auto& t : threads) {
//         t.join();
//     }

```

```
//      // 注意：使用 new 创建的实例，需要手动 delete 释放内存，
//      // 或者通过 RAII 机制（如智能指针）管理。
//      // 对于单例，通常可以不显式 delete，让操作系统在程序结束时回收。
// }
```

优点：

线程安全： 由 `std::call_once` 保证。

懒加载： 只有在第一次调用 `getInstance()` 时才创建实例。

性能较高： 初始化完成后，后续访问没有同步开销。

缺点： 相较于静态局部变量，代码略显冗长，且需要手动管理 `instance` 的生命周期（如果使用 `new`）。

4. 懒汉式 - 双重检查锁定 (DCL) + `std::atomic`

在C++11之前，DCL由于内存模型和指令重排的问题，实现起来非常棘手。C++11引入了 `std::atomic` 后，可以更可靠地实现DCL，但通常不如静态局部变量或 `std::call_once` 简单和推荐。

```
#include <iostream>
#include <atomic>
#include <mutex>
#include <thread>
#include <vector>

class SingletonDCL {
private:
    SingletonDCL() {
        std::cout << "SingletonDCL instance created." << std::endl;
    }
    SingletonDCL(const SingletonDCL&) = delete;
    SingletonDCL& operator=(const SingletonDCL&) = delete;

    // 必须使用 std::atomic 来保证线程安全和内存可见性
    static std::atomic<SingletonDCL*> instance;
    static std::mutex mtx;
```

```

public:
    static SingletonDCL* getInstance() { // 注意这里返回指针，方便原子操作
        SingletonDCL* tmp =
instance.load(std::memory_order_relaxed); // 第一次检查，不带锁，性能高
        std::atomic_thread_fence(std::memory_order_acquire); // 内存
        栅栏，保证后续读操作可见性
        if (tmp == nullptr) {
            std::lock_guard<std::mutex> lock(mtx);
            tmp = instance.load(std::memory_order_relaxed); // 第二
            次检查，在锁内
            if (tmp == nullptr) {
                tmp = new SingletonDCL();
                std::atomic_thread_fence(std::memory_order_release);
                // 内存栅栏，保证写操作可见性
                instance.store(tmp, std::memory_order_relaxed);
            }
        }
        return tmp;
    }

    void doSomething() {
        std::cout << "SingletonDCL doing something." << std::endl;
    }
};

// 静态成员变量的定义和初始化
std::atomic<SingletonDCL*> SingletonDCL::instance = nullptr;
std::mutex SingletonDCL::mtx;

// 使用示例
// void thread_func_dcl() {
//     SingletonDCL* s = SingletonDCL::getInstance();
//     s->doSomething();
// }
// int main() {
//     std::vector<std::thread> threads;
//     for (int i = 0; i < 5; ++i) {
//         threads.emplace_back(thread_func_dcl);
//     }
//     for (auto& t : threads) {
//         t.join();
//     }
// }

```

```
//    }  
//    // 同样，需要手动管理内存  
//    // delete SingletonDCL::getInstance(); // 谨慎操作，仅在确认不再  
//    使用时进行  
// }
```

优点：

线程安全： 通过 `std::atomic` 和互斥锁保证。

懒加载： 只有在第一次调用时才创建实例。

性能较高： 初始化完成后，后续访问大部分不涉及锁。

缺点：

实现复杂： 需要深入理解C++内存模型和 `std::atomic` 的 `memory_order`，容易出错。

通常不如静态局部变量或 `std::call_once` 简洁和推荐。